

WeatherPal Android engineering review

Architecture patterns structure and correctness

Reviewed October 8, 2026 | Commit cca2d44c9359

Assessment

WeatherPal has a sound architecture for a small native Android app. Its separation is substantive: the UI consumes repository contracts, business rules are independent of Android, and Room is the authoritative forecast stream. Keep this design. Three confirmed recovery defects need targeted fixes before the app can be considered robust under failure and rapid navigation.

The assessment covers the nine requested engineering criteria, with source inspection, fresh execution of the existing tests, lint, and three temporary diagnostic probes. Visual design and the real-world validity of activity scoring are outside this review.

Criterion	Assessment	Basis
Clean native architecture	Meets	Compose, MVVM, repositories and cohesive data packages; one module is proportionate.
UI domain API and state separation	Meets	No DTO, DAO or HTTP dependency in either ViewModel; pure scoring and explicit mapping.
Platform appropriate implementation	Mostly meets	Lifecycle-aware collection, saved state and desugaring; minimum-API coverage remains open.
Explicit state modeling	Mostly meets	Sealed loading/result/error and refresh states; forecast identity and selection are duplicated.
Testable ViewModels and presentation	Meets	Constructor-injected repositories, Clock, dispatchers and SavedStateHandle; deterministic tests.
Mockable API and data layer	Meets	Repository, remote and store interfaces; real Retrofit exercised with MockWebServer.
Robust error handling	Partially meets	Typed failures and transactional cache safety; cancellation, observer recovery and cooldown gaps.
Meaningful unit tests	Mostly meets	28 behavior-focused unit tests and 10 connected tests; recovery combinations are missing.
README decisions and tradeoffs	Meets	Reproducible setup, architecture, contracts, omissions and verification references are explicit.

Meets means supported by the reviewed implementation and evidence. Mostly meets means the approach is sound with a material limitation. Partially meets means a demonstrated behavior contradicts the intended contract.

Actual dependency flow

Compose screens send actions to ViewModels and render their StateFlow. ViewModels depend on domain repository interfaces. Data implementations coordinate Retrofit and Room, map external models to domain values, and publish committed snapshots. AppContainer is the composition root. Domain scoring depends on domain values and java.time.

Source: domain/repository/Repositories.kt lines 6-16; di/AppContainer.kt lines 24-64; data/repository/CachedWeatherRepository.kt lines 33-79

Recovery and cancellation findings

P2 means a correctness issue to fix in the next implementation pass. The triggers below were reproduced with deterministic JVM probes. No P0 or P1 issue was established within this review's scope.

F1 P2 A cancelled shared refresh can strand a new screen

Trigger: open an uncached city, leave while its refresh is active, then reopen the same city before the old refresh finishes cancellation. The probe deliberately delayed the old remote call's cancellation cleanup. After it was released, the new screen remained InitialLoading with RefreshStatus.Refreshing. Only one remote call occurred; the new refresh had joined the cancelled one.

CachedWeatherRepository stores one CompletableDeferred per city. A later caller awaits that deferred. Cancelling the owning request cancels the deferred and therefore throws CancellationException in the later caller, even when that caller is still active. ForecastViewModel exits without transitioning its refresh state. The initial-loading screen provides no retry control, so the user must leave and reopen again.

Fix: define ownership explicitly. For this app's navigation-cancellation policy, an active caller should discard a cancelled prior flight and start or join a live replacement. Preserve cancellation when the caller itself is cancelled. A reference-counted shared request is another option if multiple active consumers are intended. Ensure an active screen cannot retain Refreshing after its work terminates.

Regression: cover owner cancellation with a live waiter, delayed cleanup during same-city re-entry, and waiter-only cancellation. This is a proven orchestration defect under the probe's controlled timing, not a claim that every Retrofit cancellation or normal back navigation reproduces it.

Source: data/repository/CachedWeatherRepository.kt lines 43-89; ui/forecast/ForecastViewModel.kt lines 81-103; ui/forecast/ForecastViewModel.kt lines 168-199; ui/forecast/ForecastScreen.kt lines 42-65

F2 P2 Recent city observation has no recovery path

SearchViewModel catches a cache read exception, records cacheFailure, and lets its one lifetime collection finish. No action restarts that collection. The probe made the first read fail and subsequent reads succeed; submitting a search and invoking retry still left the read count at one, the shortcuts empty and the storage error present.

Existing shortcuts remain visible if a later read fails, which is useful, but they then stop updating. The displayed storage message asks the user to retry; the search retry method only retries geocoding. Rotation retains the same ViewModel and therefore does not repair the terminated observer.

Fix: add a distinct retry action for cached-city observation, or use bounded retry for genuinely transient read failures. Retain prior shortcuts while recovering and clear cacheFailure on the first successful emission. Avoid an unbounded retry loop for a persistent database or schema failure.

Source: ui/search/SearchViewModel.kt lines 42-64; ui/search/SearchScreen.kt lines 197-204; ui/Presentation.kt lines 34-44

Rate limiting and structural improvements

F3 P2 Reopening a forecast bypasses Retry After

The forecast cooldown exists only in the current `RefreshStatus.Failed`. Leaving clears that state, and opening a city builds a fresh `ForecastState`. Its initial cache emission automatically calls `refresh`. The probe returned `RATE_LIMITED` with a deadline 60 seconds ahead: a direct refresh was correctly blocked, but leaving and immediately reopening the same city issued a second request without advancing the `Clock`.

This violates the specification's treatment of opening a cached city as an explicit retry. It also means another city can bypass a limit on the same forecast service. The current unit test verifies search retry blocking while the error stays in state; it does not verify cooldown across forecast navigation.

Fix: retain the cooldown at the repository or remote-service boundary and enforce it for every request entry point. Scope it to the provider's limiting policy; geocoding and forecasting use separate hosts. Return the remaining `retryAt` so the UI can continue showing a countdown. Test same-city reopen, a different city on the same service, and the exact deadline.

Source: `ui/forecast/ForecastViewModel.kt` lines 100-123; `ui/forecast/ForecastViewModel.kt` lines 168-172; `ui/forecast/ForecastViewModel.kt` lines 249-256; `docs/implementation-specification.md` lines 590-595

P3 Reduce duplicated forecast state

`ForecastState` contains nullable city and content, while each content variant carries another city. `selectedDate` also lives in `Ready`, a private selection variable and saved state. The code coordinates these today, but its types permit mismatched identities and closed screens with content. Consolidate authoritative screen identity and selection, make `Closed` explicit, and serialize named saved-state fields through a small adapter. Keep the useful separation between content and refresh progress.

Source: `ui/forecast/ForecastViewModel.kt` lines 18-78; `ui/forecast/ForecastViewModel.kt` lines 129-165

P3 Make execution and dependency boundaries clearer

`AppDispatchers` lives inside the `DI` package, which makes `ViewModels` import the `composition` package. Only `main` is used; `io` and `computation` currently promise no execution boundary. Remote mapping and forecast scoring resume on the caller context, normally `Main`. This bounded workload is small, and no jank was measured. Move the dispatcher contract to a neutral package; use `computation` for growing work or remove unused fields. The discarded `rank` call in `CachedWeatherRepository` is redundant with remote validation and presentation ranking.

Source: `di/AppContainer.kt` lines 18-22; `data/remote/Api.kt` lines 262-279; `data/repository/CachedWeatherRepository.kt` lines 64-65

P3 Split responsibilities when these files change

`Api.kt` combines DTOs, services, client construction, error translation, mapping and a repository implementation. Splitting those responsibilities would improve navigation and review. `RoomForecastStore`'s raw SQL meets the selective-column-write requirement and binds values safely, but column names bypass `Room`'s compile-time query checking. Retain its trigger-based tests when evolving the schema. These are maintenance observations, not evidence of an injection vulnerability or a need for more modules.

Source: `data/remote/Api.kt` lines 22-179; `data/local/RoomForecastStore.kt` lines 50-55

Verification and test quality

Fresh verification passed on October 8, 2026 using Android Studio's bundled JDK and the Medium_Phone emulator on Android 16, API 36. The original suites passed after removing the probes. The debug app was reinstalled after connected-test cleanup.

```
./gradlew.bat :app:testDebugUnitTest :app:lintDebug `
:app:connectedDebugAndroidTest
```

Existing suite	Tests	Outcome
ScoringTest	7	Passed
RemoteTest	8	Passed
RepositoryTest	3	Passed
ViewModelTest	10	Passed
Room connected tests	5	Passed
Compose connected tests	5	Passed
Android lint	26 warnings	0 errors

The tests provide useful evidence

Scoring tests check boundary values, worked examples, ordering, missing inputs, complementary scores, and 23/25-hour DST days. Network tests inspect encoded request parameters, filtering, malformed responses, transport recovery and cancellation. ViewModel tests use injected Clock, dispatchers and handwritten fakes to verify stale-result rejection, saved-state reconstruction, refresh preservation and queued rollover.

Room instrumentation is particularly valuable: SQLite triggers detect unintended row or column writes, force a failure after eviction to verify rollback, and check coherent snapshots, persistence and concurrent capacity limits. Compose tests verify disclosure state through ranking changes, restored composition state and date navigation. These assertions cover behavior rather than merely checking that functions were called.

Additional diagnostic evidence

A separate fresh unit run passed all 28 existing tests plus three temporary probes. Those probes asserted the current defective outcomes for F1, F2 and F3; passing them confirms the observations, not correct product behavior. They were removed from app sources after execution. Their source and XML results are retained under the ignored tmp/architecture-review directory for local reproduction.

Limits of the evidence

No coverage percentage was measured. Add recovery tests for failed repository flows, owner versus waiter cancellation, rate limits across navigation, and pure presentation formatting. SavedStateHandle reconstruction and Compose restoration tests do not establish real process-death recovery end to end. API 24/25, physical devices, full TalkBack behavior, time changes during an already-scheduled midnight timer and release/minified behavior remain unverified.

Lint's 26 warnings concern dependency/tool update availability and kapt usage. They are maintenance signals; their count does not establish a functional defect. The review did not perform a dependency security audit.

Source: test/ui/ViewModelTest.kt lines 90-409; test/data/RepositoryTest.kt lines 88-179; androidTest/CacheInstrumentedTest.kt lines 45-153; androidTest/ActivityCardInstrumentedTest.kt lines 37-131

Platform fit documentation and next steps

The native implementation is appropriate

The app uses `ComponentActivity`, `Compose` and `Material 3`, lifecycle-aware `StateFlow` collection, `ViewModel` lifetimes, `SavedStateHandle`, saveable component state and explicit `Back` handling. `Room` suspending `DAOs` and `Retrofit` suspending services avoid blocking database/network I/O in the UI. City-local `java.time` calculations are desugared for min SDK 24. `INTERNET` is the only requested permission. State and action parameters keep the screens independent of `ViewModel` construction.

Two screens do not require a navigation framework, and this scope does not require `Hilt`, `WorkManager`, a use-case wrapper for each repository call, or mandatory multi-module Clean Architecture. The English-only interface, hardcoded strings and activity-scoped `ViewModels` are manageable choices at this size. Adopt string resources and clearer destination ownership as localization and navigation grow. Package separation is conventional rather than compiler-enforced.

Source: `MainActivity.kt` lines 34-100; `app/build.gradle.kts` lines 12-43; `app/src/main/AndroidManifest.xml` lines 5-24

The README explains meaningful decisions

`README.md` clearly documents build prerequisites, runtime JDK versus bytecode targets, package responsibilities, manual DI, `Room` as the source of truth, public API contracts, scoring, cache transactions, tests and production omissions. It links a detailed verification record that separates fixtures from live observations. This satisfies the requested documentation criterion.

Improve it after the fixes: explain shared-refresh ownership and observer recovery; document where cooldowns live; add a compact dependency diagram and a short smoke-test recipe. Its claim that `Retry-After` disables retries currently needs the navigation qualification from F3. State the current test total of 28 near the commands so readers do not have to reconstruct historical counts in the verification log. Preserve the useful disclosure of single-module/manual-DI tradeoffs.

Source: `README.md` lines 9-45; `README.md` lines 86-114

Recommended implementation order

1. Fix F1 and add owner/waiter and same-city re-entry regressions. An active screen must always reach `Ready` or a recoverable error when refresh work ends.
2. Fix F2 with a cache-specific recovery action and tests proving that shortcuts resume updating after a successful read.
3. Fix F3 at the data boundary and test cooldown preservation across navigation, cities and deadline expiry.
4. Consolidate forecast state and saved-state serialization while keeping content and refresh status independently modeled.
5. Add automated unit/lint checks to CI, then broaden device/process-death coverage as release scope requires. Update the README to describe the resulting behavior.

Review basis

Source snapshot: `cca2d44c93597b875d2982612d1d89e2ab622662`. Source paths and lines refer to that snapshot. Repository root: `C:/Users/linco/AndroidStudioProjects/WeatherPal`. Production source files and existing tests were left unchanged.

[Android architecture recommendations](#)

[Android domain layer guidance](#)

[Android offline-first data guidance](#)

[Kotlin Flow retry semantics](#)